Performance budget (item 11)

What is measured, the limit, and where it is checked. Browser numbers come from headless Chromium with software rendering, which is slower than a desktop browser with a GPU; the limits are set for that browser.

Budget

What	Limit	Checked by
App page (HTML, one file)	< 300 KB	`app/test_ui.cjs 1p`
Graph usable after navigation (local server)	< 3 s	`app/test_ui.cjs 1p`
Frame (rotate one step), full graph, with and without the field view	median < 40 ms	`app/test_ui.cjs 1p`
Observer-lens route computed in the page	median < 40 ms	`app/test_ui.cjs 1p`
JS heap after load	< 150 MB	`app/test_ui.cjs 1p`
Server CPU per flat route (in-process)	median < 15 ms	`server/test/mcp.test.mjs`
Server CPU per lens route (in-process)	median < 60 ms	`server/test/mcp.test.mjs`
Live lens route with dense	no error 1102 (Cloudflare resource limit)	live check after each deploy

Measurements

Before (server 6.5.1 / app 1.10, live, measured from Claude's cloud workspace)

What	Value
App HTML	93 KB over the wire (gzip), 247 KB raw
graph.json	946 KB over the wire, 3.5 MB raw; 156–228 ms to fetch
DOMContentLoaded	263 ms first load, 47 ms cached
Graph usable	1,287 ms first load, 1,159 ms cached
Frame, full graph	median 13.1–16.5 ms, p90 20.5–28.4 ms
Frame, field view	median 15.2–15.4 ms, p90 17.1–19.1 ms
Lens route in the page	56–62 ms
JS heap	54–61 MB
/route end to end (includes the workspace's proxy)	flat 0.12–0.88 s, lens 0.20–1.10 s

The profile of a page load showed three avoidable costs: the agent ring layout re-counted every concept's skills for every concept (about 1.5 M filter steps per call, ~160 ms), each agent's angle scanned all edges (~55 ms), and the gate list sorted all 117 servers for every agent (~45 ms).

After (app 1.11, local measurements; same results checked node by node)

What	Before	After
Graph usable (local server)	991–1,027 ms	737–776 ms
Lens route in the page	67–71 ms	15–17 ms (10.9 ms in the test run)
Frame, full graph / field view	—	15.1 / 15.7 ms

What	Before	After
Server CPU per route (in-process)	lens ~80 ms before 6.5.1	flat 2.7 ms, lens 16 ms

Every node's place, the lens chain on the reference task, the number of gates and each agent's gate count were compared between the old and new page: identical.

After, live (server 6.5.2 / app 1.11, same workspace, same script `app/perf_live.cjs`)

What	Before (6.5.1)	After (6.5.2)
Graph usable, cached load	1,159 ms	888 ms
Graph usable, first load	1,287 ms	1,522 ms (the HTML took 524 ms on the first request after the deploy, against 141 ms before: a cold edge cache, not page work)
Lens route in the page	56–62 ms	10.7–11.3 ms
Frame, full graph	median 13.1–16.5 ms	median 18.5–21.0 ms
Frame, field view	median 15.2–15.4 ms	median 18.8–22.1 ms
JS heap	54–61 MB	40–54 MB
Live lens routes with dense	—	10 of 10 returned 200 (no error 1102)

Frame times rose in this run although no drawing code changed in 6.5.2; the workspace's shared CPU varies between runs, so single runs of a few milliseconds are

noise here. All values are inside the budget.

Not yet done

- graph.json (3.5 MB raw) is the largest cost of a first load; a slimmer browser payload (field places and vectors only where the page needs them) would cut it.
- The page still computes its own legacy 48-dimension hash embedding at load (~80 ms); it only feeds the gate re-rank.